DHARMA TEJA GURRAM

Bengaluru, India +91 9182436977 gurramdharma925@gmail.com
linkedin.com/in/dharma925 github.com/dharma925 dharma925.github.io/portfolio

SUMMARY

ASIC/SoC Design Verification Engineer with hands-on experience across hardware verification, EDA workflows, and AI-driven engineering automation. Builds LLM-powered agents and multi-agent systems for regression analysis, simulation debug, workflow orchestration, and CI/CD automation, alongside experience with RTL verification, SoC bring-up, and neural-network hardware accelerators.

EDUCATION

Indian Institute of Technology (IIT) Mandi *Mandi, India*
Bachelor of Technology, Electrical Engineering (Honors) — CGPA: 8.68/10 *2020 – 2024*

Coursework: Computer Organization, Reconfigurable Computing, Computing & Data Science, Digital System Design, Digital MOS LSI Circuits, Data Science, Estimation and Detection, Probability & Statistics

TECHNICAL SKILLS

- **EDA & Verification:** Cadence Xcelium, SimVision, JasperGold (Lint/CDC), vManager, Virtuoso, Makeflow (ARMCC, ARMClang, ARMLink)
- **Programming & Scripting:** Python, C/C++ (Object-Oriented / Modeling Foundations), Tcl, Bash, Csh
- **AI/ML & Agentic Systems:** TensorFlow, Pytorch, Numpy, Scikit-learn, Claude API, LLM agent orchestration, MCP development, RAG (Vectara), Custom Multi-Agent Frameworks, Agentic Playground Design
- **Software & Cloud:** Node.js, React, Docker, AWS (EC2, S3, RDS, Lambda), Electron, OIDC SSO
- **Version Control & CI/CD:** Perforce (Helix Core), Design Sync, Git, Bitbucket, Jira, Confluence
- **Design & HDL:** SystemVerilog, Verilog, VHDL, Verilog-A

PROFESSIONAL EXPERIENCE

Design Verification Engineer | *Texas Instruments, Bengaluru, India* *Jul 2024 – Present*

- Architected and developed an LLM-powered, state-aware agentic automation stack that transforms manual VLSI workflows into autonomous CI/CD pipelines — maximizing off-hours compute utilization and significantly shrinking project timelines.
  - **Multi-Agent Workflow Playground** — plug-and-play framework for composing and experimenting with multi-agent LLM workflows over EDA flows; shipped React/Node dashboards measuring pipeline health, coverage and automation impact.
  - **Autonomous Regression Lifecycle Manager** — hands-off agent handling workspace syncs, regression triage, local fixes and debug reporting; reclaims idle off-hours compute, saving **~2 person-months** per ~1.5-year cycle and freeing lead-engineer bandwidth.
  - **Spec-Aware Semantic-Temporal Debug Agent** — MCP-based closed-loop AI debug agent enabling natural-language querying of simulation data via a custom semantic query adapter (`simvisdbutil`/VCD) to isolate relevant signal windows and root-cause failures, cutting debug turnaround by **6x**.
  - **Workspace Health Checker** — Design Sync pre-submit gate running sanity checks; embedded LLM agent diagnoses failures and suggests fixes, eliminating post-sync breakages, conflicts and version jumps.
  - **Release-Triggered Quality Gate** — listens for release emails, auto-launches per-IP Lint/CDC checks, monitors LSF job health, and posts violation summaries to Confluence.
  - **ChatOps Workspace Orchestrator** — Webex-integrated interface letting engineers securely trigger and monitor Linux workspace operations (syncs, LSF builds) via natural language.
- Drove BU-wide adoption of AI-assisted engineering (Claude Code, custom MCP integrations) across design/verification teams.
- ARM Boot & **C-Toolchain Framework** — built the SoC's ARM-core C-compilation/boot infrastructure from scratch: full make-flow and toolchain integration (ARM CC/Link, ELF-to-hex, RAM/ROM loading) for flexible boot/firmware test-case composition, plus a SimVision toolbar for relaunch-free recompiles.
- Owned DV for critical IP blocks (ADC, OTP, DPWM, Filter) — testcase/checker/assertion development (including DV-plan-driven testcase-skeleton automation) and sanity/regression closure on every RTL release, in Xcelium/vManager stack.

Digital Design Intern | *Texas Instruments, Bengaluru, India* *Jan 2023 – Jul 2023*

- UCD3138 Verification — implemented interleaved (2/3-phase) peak-current-mode control from system-level requirements and verified it end-to-end in Xcelium/SimVision: Verilog/AMS debug, full pad-to-pad data-path verification, and bring-up to a working SoC condition with the system engineer.
- Synthesized IP blocks (I^2C, PMBus, Timer) for area/power/timing reports and netlists; authored an implementation spec from a communication-interface IP analysis.

PROJECTS

MACTrace — C++ Functional & Performance Model of a Configurable CNN Accelerator `C++17 · CMake · PyTorch · Python`

- Built a C++ functional model of a configurable CNN accelerator (MAC array, accumulator, SRAM buffer classes) mapping convolution and fully-connected layers onto tiled GEMM execution, validated against a PyTorch-trained INT8-quantized MNIST model.
- Instrumented the accelerator with performance counters (MAC utilization, cycle estimates, buffer traffic) to compare hardware configurations, measuring utilization differences between convolution and fully-connected layers on the same workload.

NeuronEase — Memristor-Based Hardware Accelerator for Neural Networks `Cadence Virtuoso · Verilog-A · TensorFlow`

- Final-year project: designed/simulated a memristor-crossbar accelerator (Stanford ReRAM, Cadence Virtuoso, Verilog-A) for analog multiply-accumulate — weights as crossbar conductances, DAC/ADC I/O; validated via a SPICE-level 2×2 matrix-multiplication demo.
- Trained a compact CNN in TensorFlow (3×3, 12-kernel conv + max-pool + dense, 20,410 INT8 params) and mapped it onto a 9×8 memristor crossbar (row/col decoders, DAC/ADC, conv controller); verified against MNIST at ~12 cycles/pixel, and separately fabricated/characterized $Cu/TiO_2/Pt$ memristive cells (VLSI Fabrication Practicum), measuring I–V resistive switching (I_{on}/I_{off} up to 10^4).

FCNN on FPGA — Fully Connected Neural Network Accelerator `Verilog · Vivado · TensorFlow · OpenCV`

- Built an FCNN in Verilog on a Zybo Z7 FPGA (Vivado) from primitives (adders, multipliers, neurons, dense layers), 16-bit fixed point, TensorFlow-trained; extended into an image-processing pipeline streaming FPGA data, convolving on-chip, results to Python (OpenCV).